Do spectral gradient updates help?

Auguste Ishema

2026-10-26

Table of contents

1 Introduction **2**

2 References **2**

3 Methodology **3**
3.1 Dataset . . . 3
3.2 Objective function . . . 3
3.3 Features . . . 4
3.4 GD Methods . . . 5
3.4.1 Classic GD with constant steps . . . 5
3.4.2 HB . . . 5
3.4.3 NAG . . . 6
3.4.4 Spectral GD . . . 6

4 Temporary Results **7**
4.1 Optimizers Params . . . 7
4.2 Results . . . 7
4.3 Todo . . . 7

1 Introduction

Let $d \in \mathbb{N}^*$ and $F : \mathbb{R}^d \to \mathbb{R}$ be a differentiable function, the gradient descent (GD) method defines sequences $(x_k)_{k\in\mathbb{N}}, (\alpha_k)_{k\in\mathbb{N}}$, such that $\forall k \in \mathbb{N};\ x_k \in \mathbb{R}^d$, $\alpha_k \in \mathbb{R}_+^*$ and

$$\forall\, k \in \mathbb{N}, \quad x_{k+1} = x_k - \alpha_k \nabla F(x_k)$$

This method is widely used for optimizing objective functions in machine learning. Several variants have been proposed:

- GD with constant steps or decreasing steps
- GD with inertia and/or acceleration: Heavy Ball (HB), Nesterov Acceleration (NAG), ...
- Stochastic GD (SGD)
- Adaptive and normalized GD methods: AdaGrad, RMSprop, Adam, Sign GD, Spectral GD, Muon, ...

In this study, we focus on the spectral GD method.

Let $(n, d) \in \mathbb{N}^* \times \mathbb{N}^*$ and a function $F : \mathcal{M}_{n,d}(\mathbb{R}) \to \mathbb{R}$, Spectral GD defines the sequences $(x_k)_{k\in\mathbb{N}}, (\alpha_k)_{k\in\mathbb{N}}$, such that $\forall k \in \mathbb{N};\ x_k \in \mathcal{M}_{n,d}(\mathbb{R})$, $\alpha_k \in \mathbb{R}_+^*$ and

$$\forall\, k \in \mathbb{N}, \quad x_{k+1} = x_k - \alpha_k U_k V_k^T$$

Where $\nabla F(x_k) = U_k \Sigma_k V_k^T$ is the SVD decomposition of $\nabla F(x_k)$ and U_k and V_k are such that

$$U_k V_k^\top = \arg \max_{\|D\|_S \leq 1} \langle \nabla F(x_k), D \rangle$$

With $\|D\|_S$ the spectral norm (the largest singular value)

The aim of this project is to determine, on a few examples, when spectral GD improves the convergence rate of optimization problems compared to other gradient descent methods.

TODO: Improve the definition of Spectral GD

2 References

- Tsanas, A. & Xifara, A. (2012). Energy Efficiency Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C51307.
- Tam Le, Université Paris Cité - LPSM, Optimization for machine learning

3 Methodology

3.1 Dataset

Tsanas, A. & Xifara, A. (2012). Energy Efficiency Dataset. UCI Machine Learning Repository. https://doi.org/10.24432/C51307.

It contains $n = 768$ buildings described by 8 features and two continuous targets:

Variable	Description	Type
X_1	Relative compactness	geometric
X_2	Surface area	geometric
X_3	Wall area	geometric
X_4	Roof area	geometric
X_5	Overall height	geometric
X_6	Orientation	discrete
X_7	Glazing area	geometric
X_8	Glazing area distribution	discrete
Y_1	Heating load	target
Y_2	Cooling load	target

Since we are interested in the optimization dynamics rather than in predictive performance, we do not split the data into training and test sets.

3.2 Objective function

Definition

Let

- $n, d, p \in \mathbb{N}^*$
- $X \in \mathcal{M}_{n,d}(\mathbb{R})$ features
- $Y \in \mathcal{M}_{n,p}(\mathbb{R})$ targets
- $\forall \beta = (\beta_{i,j})_{(i,j)\in[\![1,d]\!]\times[\![1,p]\!]} \in \mathcal{M}_{d,p}(\mathbb{R}),\ \|\beta\|_F^2 = \sum_{i,j} \beta_{i,j}^2$ the Frobenius norm

The OLS linear regression objective function is such that $\forall \beta \in \mathcal{M}_{d,p}(\mathbb{R})$

$$F(\beta) = \frac{1}{2np}\|X\beta - Y\|^2$$

F is twice differentiable and

$$\nabla F(\beta) = \frac{1}{np} X^T(X\beta - Y) \quad \text{and} \quad \nabla^2 F(\beta) = \frac{1}{np} X^T X$$

Since $\forall \beta \in \mathcal{M}_{d,p}(\mathbb{R}), \nabla^2 F(\beta) \succeq 0$, F is convex therefore it has a global minimizer β^* such that

$$\nabla F(\beta^*) = \frac{1}{np} X^T(X\beta^* - Y) = 0$$

Exact solution

Therefore if $X^T X$ is inversible (no multicollinearity), β^* is such that:

$$\beta^* = (X^T X)^{-1} X^T Y$$

Approximate of solution by gradient descent

When $X^\top X$ is not invertible or when $X^\top X$ is too large to inverse easily, an approximation of β^* obtained by GD can still be satisfying.

One can notice that:

- $\forall \beta \in \mathcal{M}_{d,p}(\mathbb{R}),\ \nabla^2 F(\beta) \succeq 0$
- F is L-smooth with $L = \frac{\lambda_{\max}(X^\top X)}{np}$
- If $\lambda_{\min}(X^\top X) > 0$, F is μ-strongly convex with $\mu = \frac{\lambda_{\min}(X^\top X)}{np}$
- $\mu > 0$ so $\kappa = \frac{L}{\mu} = \frac{\lambda_{\max}(X^\top X)}{\lambda_{\min}(X^\top X)}$

3.3 Features

Describes

	count	mean	std	min	25%	50%	75%	max
X1	768.0	0.764167	0.105777	0.62	0.6825	0.75	0.8300	0.98
X2	768.0	671.708333	88.086116	514.50	606.3750	673.75	741.1250	808.50
X3	768.0	318.500000	43.626481	245.00	294.0000	318.50	343.0000	416.50
X4	768.0	176.604167	45.165950	110.25	140.8750	183.75	220.5000	220.50
X5	768.0	5.250000	1.751140	3.50	3.5000	5.25	7.0000	7.00
X6	768.0	3.500000	1.118763	2.00	2.7500	3.50	4.2500	5.00
X7	768.0	0.234375	0.133221	0.00	0.1000	0.25	0.4000	0.40
X8	768.0	2.812500	1.550960	0.00	1.7500	3.00	4.0000	5.00
Y1	768.0	22.307201	10.090196	6.01	12.9925	18.95	31.6675	43.10
Y2	768.0	24.587760	9.513306	10.90	15.6200	22.08	33.1325	48.03

Correlation

```
df_corrs = pd.DataFrame(X_raw, columns=cols_X_raw).corr().round(4)
df_corrs.abs().style.background_gradient(cmap ='viridis', axis=None)
```

0.3s

	X1	X2	X3	X4	X5	X6	X7	X8
X1	1.000000	0.991900	0.203800	0.868800	0.827700	0.000000	0.000000	0.000000
X2	0.991900	1.000000	0.195500	0.880700	0.858100	0.000000	0.000000	0.000000
X3	0.203800	0.195500	1.000000	0.292300	0.281000	0.000000	0.000000	0.000000
X4	0.868800	0.880700	0.292300	1.000000	0.972500	0.000000	0.000000	0.000000
X5	0.827700	0.858100	0.281000	0.972500	1.000000	0.000000	0.000000	0.000000
X6	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
X7	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.213000
X8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.213000	1.000000

Feature correlation and conditioning

The variables (X_1, X_2) and (X_4, X_5) are strongly correlated. Hence the gram matrix $X^\top X$ is not inversible and it is badly conditioned.

```
G = X.T @ X
cond_nb = np.linalg.cond(G)
rank_g = np.linalg.matrix_rank(G)
pd.DataFrame([[rank_g, cond_nb]], columns=['rank_G', 'kappa_G'])
```

0.0s

	rank_G	kappa_G
0	7	1.524989e+16

Keeping them together would make $X^\top X$ nearly singular and would blur the convergence analysis. To start we will only keep the following features: `cols_X` $= \{X_1, X_3, X_6, X_7, X_8\}$ (hence $d = 5$), to observe convergence rates in a simple setting, with an objective function which Hessian is well conditioned.

	X1	X3	X6	X7	X8
X1	1.000000	0.203800	0.000000	0.000000	0.000000
X3	0.203800	1.000000	0.000000	0.000000	0.000000
X6	0.000000	0.000000	1.000000	0.000000	0.000000
X7	0.000000	0.000000	0.000000	1.000000	0.213000
X8	0.000000	0.000000	0.000000	0.213000	1.000000

```
G = X.T @ X
cond_nb = np.linalg.cond(G)
rank_g = np.linalg.matrix_rank(G)
pd.DataFrame([[rank_g, cond_nb]], columns=['rank_G', 'kappa_G'])
```

0.0s

	rank_G	kappa_G
0	5	1.541181

Standardization

The retained features are centered and reduced (z-score), column by column. So that each column has zero mean and unit variance. This puts all features on the same scale and makes the intercept unnecessary in the OLS linear model. it also ensures that the curvature of the objective reflects the correlation structure of the data rather than the arbitrary units of the features. The targets are left unchanged.

In summary, the optimization problem is defined by $X \in \mathbb{R}^{n \times d}$ with $n = 768, d = 5$, and $Y \in \mathbb{R}^{n \times p}$ with $p = 2$.

3.4 GD Methods

3.4.1 Classic GD with constant steps

```
spec_gram_x = np.linalg.eigh(G).eigenvalues
lambda_min, lambda_max = np.min(spec_gram_x), np.max(spec_gram_x)
display(pd.DataFrame([[lambda_min, lambda_max]], columns=['lambda_min', 'lambda_max']))
```

0.0s

	lambda_min	lambda_max
0	604.443478	931.556522

Since the objective function F is μ-strongly convex and L-smooth. We can use the following optimal constant step size

- $\alpha^* = \frac{2}{\mu + L}$
- Convergence rate: $O\left(\left(\frac{\kappa-1}{\kappa+1}\right)^{2k}\right)$ and $F(x_k) - F^* \leq \left(\frac{\kappa-1}{\kappa+1}\right)^{2k} \frac{LR^2}{2}$ with $R = \|x_0 - x^*\|^2$ and for a given $\epsilon \in \mathbb{R}^{+*}$,

$$\left(\frac{\kappa-1}{\kappa+1}\right)^{2k} \frac{LR^2}{2} \leq \epsilon \quad \iff \quad k \geq \frac{log\left(\frac{2\epsilon}{LR^2}\right)}{2log\left(\frac{\kappa-1}{\kappa+1}\right)}$$

3.4.2 HB

The HB method adds an inertia term the the classic GD method It defines the sequence $(x_k)_{k \in \mathbb{N}}$ such that $x_1 = x_0 - \alpha \nabla F(x_0)$ and

$$\forall\, k \in \mathbb{N}, \quad x_{k+1} = x_k - \alpha \nabla F(x_k) + \beta(x_k - x_{k-1})$$

Optimal params

Since F is μ-strongly convex and L smooth and $\kappa = \frac{L}{\mu}$, the optimal params are such that

- $\alpha^* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}$
- $\beta^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2$

- For a given $\delta > 0$ there exists $C_\delta > 0$ such that for all $k \in \mathbb{N}$

$$F(x_k) - F^* \leq C_\delta \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} + \delta \right)^{2k} \|x_0 - x^*\|^2$$

  Therefore the convergence rate is

$$O\left(\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^{2k} \right)$$

  And for a given $\epsilon \in \mathbb{R}^{+*}$ and $R = \|x_0 - x^*\|^2$

$$C_\delta \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} + \delta \right)^{2k} R^2 \leq \epsilon \quad \iff \quad k \geq \frac{log(\frac{\epsilon}{C_\delta R^2})}{2log(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} + \delta)}$$

Todo review

3.4.3 NAG

Nesterov Acceleration Gradient Method like HB adds an inertia term to the classic GD method, However it changes the point in which the gradient is evaluated

It defines the sequence $(x_k)_{k \in \mathbb{N}}$ such that $x_1 = x_0 - \alpha \nabla F(x_0)$ and

$$\forall\, k \in \mathbb{N}, \quad x_{k+1} = x_k - \alpha \nabla F(x_k + \beta_{k-1}(x_k - x_{k-1})) + \beta_{k-1}(x_k - x_{k-1})$$

Optimal params

Since F is μ-strongly convex and L smooth and $\kappa = \frac{L}{\mu}$, the optimal params are such that

- $\alpha^* = \frac{1}{L}$
- $\forall k \in \mathbb{N}^*,\ \beta_k = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$
- With $R = \|x_0 - x^*\|$,

$$\forall k \in \mathbb{N}^*, \quad F(x_k) - F^* \leq \frac{L + \mu}{2} \left(1 - \frac{1}{\sqrt{\kappa}} \right)^k R^2$$

  Therefore for a given $\epsilon \in \mathbb{R}^{+*}$, $R = \|x_0 - x^*\|^2$

$$\frac{L + \mu}{2} \left(1 - \frac{1}{\sqrt{\kappa}} \right)^k R^2 \leq \epsilon \iff k \geq \frac{log\left(\frac{2\epsilon}{(L+\mu)R^2}\right)}{log\left(1 - \frac{1}{\sqrt{\kappa}}\right)}$$

Todo review

3.4.4 Spectral GD

Todo: define optimal params for Spectral GD

4 Temporary Results

4.1 Optimizers Params

	lr_gd	epsilon	epochs_gd
0	1.0	1.000000e-10	7

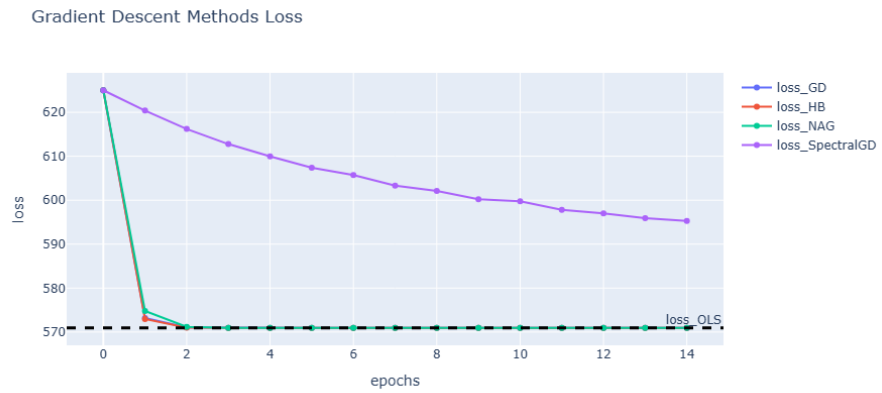
	lr_hb	mom_hb	delta	epsilon	epochs_hb
0	1.011603	0.011603	1.000000e-10	1.000000e-10	5.16682

	lr_nag	mom_nag	epsilon	epochs_nag
0	0.824427	0.107718	1.000000e-10	14.062477

	lr_spectral_gd	mom_spectral_gd
0	1.0	0

Epochs: 15

4.2 Results



Loss function last values

	OLS	GD	HB	NAG	SpectralGD
0	570.946472	570.946472	570.946472	570.946472	595.285095

Distance between weights

	GD	HB	NAG	SpectralGD
0	2.366851e-12	2.901281e-12	2.103063e-12	5.817387

One can observe that GD, methods converge quite fast in a couple iterations.

Which the expected behavior since our problem is well conditioned.

Moreover they are quite close to the optimal solution obtained by OLS.

However with the current params Spectral GD is quite slow to converge.

4.3 Todo

- Review how to set optimal params for Spectral GD
- Add other GD methods
- Test other problems with bad conditioning
- Test other objective functions